Inside the Spec

Realistic Scenarios. Working Artifacts.
Decisions That Matter.

API не ответил вовремя. Платёж прошёл?

Одно пополнение. Неизвестный результат.
Спецификация дальнейших действий.

CASE 01 · PAYMENT TIMEOUT

€50. Достоверного ответа нет.

Пользователь пополняет баланс на €50. Провайдер мог принять платёж до тайм-аута запроса.

Нет ответа со статусом и ID операции? Повторить POST с прежним ключом в пределах клиентского бюджета. После ответа со статусом либо ID кнопка «Повторить» становится «Проверить статус» через GET.

> **Тайм-аут не доказывает отказ платежа.**

Условия кейса: один провайдер; конкурентная идемпотентность 24 часа; lookup по стабильному reference; неизменные финалы; инструмент без действий пользователя. €50 не означает карточный платёж в ЕС или освобождение от SCA.

Три связанных измерения состояния.

payment_status	SUCCEEDED	Успех платежа подтверждён.
credit_status	REVIEW_REQUIRED	Нет разрешённой попытки; зачисление заблокировано.
review_status	OPEN	Открыт SECURITY_HOLD, запрещающий новую отправку.

Источник причин: review_items. review_status и blocking_review_reasons вычисляются; credit_status отражает ход зачисления и подтверждения. Открытый разбор не всегда означает hold.

Допустимые сочетания и границы.

Полный enum credit_status: NOT_REQUESTED, PENDING, DISPATCHING, OUTCOME_UNKNOWN, REVIEW_REQUIRED, POSTED.

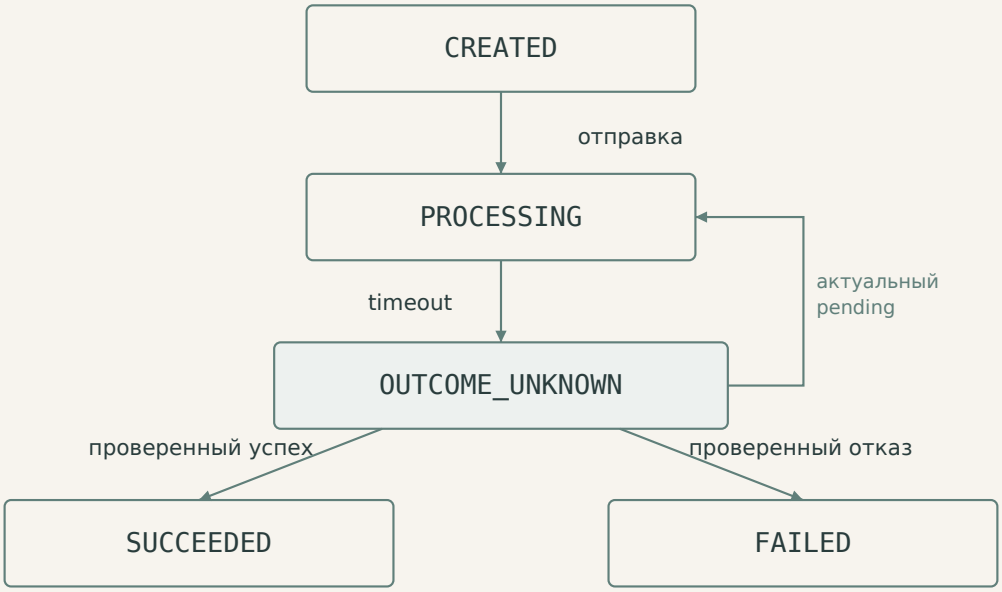
Платёж / зачисление	Ограничение
Nonfinal / NOT_REQUESTED	До успеха платежа зачисления нет.
FAILED / NOT_REQUESTED	FAILED + POSTED запрещено.
SUCCEEDED / PENDING	Нет блокировки и неизвестной доставки.
SUCCEEDED / REVIEW_REQUIRED	Есть блокировка; разрешённые попытки уже не могут создать проводку.
SUCCEEDED / DISPATCHING	Разрешённая попытка активна; возможен поздний hold.
SUCCEEDED / OUTCOME_UNKNOWN	Попытка могла создать проводку; возможен hold.
SUCCEEDED / POSTED	Проводка подтверждена, даже при разборе.

> **RESULT_OVERDUE не блокирует зачисление. SECURITY_HOLD запрещает новые отправки.**

Другие сочетания payment/credit запрещены. OPEN + PENDING допустимо только при неблокирующих разборах. Hold не стирает DISPATCHING, неизвестное зачисление или POSTED.

Не скрывайте неизвестность.

На схеме показано восстановление после неоднозначного create. Состояния отражают то, что известно нашему сервису.



Также: PROCESSING → проверенный SUCCEEDED / FAILED; CREATED → FAILED при окончательном локальном отказе до отправки. Полная таблица в спецификации.

> **Ни NOT_FOUND, ни окончание окна не означают FAILED.**

Любое проверенное PENDING провайдера сохраняет provider_operation_seen=true. Новый CREATE больше не разрешается, даже после NOT_FOUND.

Связь успеха, разбора и зачисления.

Триггер	Локальный атомарный результат
Подтверждён успех платежа	SUCCEEDED + проверка допуска в одной транзакции платёжной БД.
Нет блокировки зачисления	PENDING + CreditRequested(payment_id)
Открыта блокирующая причина	REVIEW_REQUIRED
Разрешение отправки	DISPATCHING + delivery attempt
Неоднозначная доставка	credit_status = OUTCOME_UNKNOWN
Подтверждена проводка баланса	POSTED

Проводка создаётся позже отдельной транзакцией БД баланса. Повтор успеха не сбрасывает зачисление. Hold после разрешения запрещает новые попытки, но не отзывает разрешённую.

Легенда ID: PA: предпосылка провайдера; ST: переход платежа; IC/IM: правило/матрица идемпотентности; CR: зачисление; AC: проверочный сценарий; RC: сверка. M, WR, RP, WH, KEY: модель, воркеры, часы восстановления, webhook и ключи.

Одно намерение. Те же ID.

Идентификатор	Что он обозначает
client_idempotency_key	Одно намерение; связь хранится весь срок операции, включая unknown.
payment_id	Одна внутренняя платёжная операция.
provider_idempotency_key	Та же операция провайдера при повторах create.
attempt_id	Один исходящий вызов CREATE или LOOKUP.

> **Тот же клиентский ключ + те же параметры → существующая операция.**

Область ключа: пользователь + тип операции + ключ. Fingerprint: нормализованные сумма, валюта, инструмент.

Ключ, fingerprint и операция сохраняются атомарно. Уникальность обеспечивается ограничением хранилища.

Опишите повторный запрос.

Условие	Поведение API
Те же ключ и данные; любое нефинальное состояние	202 · Полный PaymentStatus. Далее GET.
Тот же ключ, другие данные	409 · IDEMPOTENCY_KEY_REUSED.
Тот же ключ; платёж финальный	200 · Прежний результат; статус зачисления отдельно.
Новый ключ; совпал незавершённый платёж	409 · Исходный PaymentStatus + SHOW_EXISTING_PAYMENT.
Нельзя надёжно сохранить операцию	503 · Внешний вызов требует подтверждённой записи.

> **Повтор клиента сам по себе не запускает новый create провайдеру.**

HTTP кейса: 409 при изменении данных, 202 при обработке; IETF draft-07: 422/409. Ключ: весь срок операции и ≥30 дней после полного разрешения.

После 202 кнопка использует GET.

POST возвращает 202 для нефинального платежа, 200 для финального. Во фрагменте DTO опущены поля credit/review с предыдущих страниц.

```
payment_id: pay-001
payment_status: PROCESSING
status_url: /payments/pay-001
blocking_review_reasons: []
next_action: WAIT
poll_after_seconds: 5
retry_post: false
```

> **Нет ответа и ID: максимум один автоматический POST-повтор с прежним ключом. Есть ответ со статусом или ID: «Проверить статус», только GET.**

GET возвращает 200 и для известных нефинальных операций. Опрос: не чаще раза в 5 секунд, максимум 60 секунд; затем ручное обновление. Guard возвращает полную исходную операцию и SHOW_EXISTING_PAYMENT.

Блокирующий разбор или передача операционистам → CONTACT_SUPPORT. Неблокирующий OPEN не означает hold. HTTP 200 не подтверждает успех платежа или зачисления.

Одни часы, ограниченный create.

```
T0 = persisted recovery_started_at
LOOKUP at T0 + 15s / 60s / 300s / 900s / 3600s
CREATE / LOOKUP timeout: 10s
CREATE budget: initial + 1 retry
```

Это абсолютные смещения, не интервалы между вызовами. В T0+3600s независимый deadline открывает разбор и запрещает CREATE до последнего lookup. Рестарт: одна актуальная проверка, без пачки.

> **Основная защита после неоднозначного CREATE: прежний provider key и конкурентная идемпотентность. NOT_FOUND лишь допускает проверку повтора.**

Повтор без lookup требует доказанного непринятия всех прошлых попыток. Всегда проверяются бюджет, блокировки, provider_operation_seen=false и сроки. Cutoff 23 часа: внешняя страховка; обычно раньше действует часовой лимит.

Воркеры подхватывают сохранённую работу.

Сохранённое состояние	Владелец / действие
CREATED	Задание создаётся с платежом. Watchdog каждые 5s; lease 30s.
CREATED, возраст ≥60s, нет резерва	Watchdog закрывает неначатое задание под блокировкой: FAILED, guard снят, ключ сохранён.
PROCESSING, воркер потерян	Watchdog сохраняет бюджет, отмечает unknown, назначает lookup. Без слепого CREATE.

Старый воркер не резервирует попытку после закрытия. Зарезервированная попытка могла быть отправлена. Дежурный Payments отвечает за очередь, Payment Operations за просрочку. Сроки требуют доступных воркеров/БД.

HTTP-код не доказывает результат.

Ответ провайдера	Решение о восстановлении
429; непринятие доказано	Учесть Retry-After. Повтор допустим, если все прежние попытки также доказанно не приняты.
429; такой гарантии нет	Результат неизвестен. До возможного повтора нужен lookup.
502 / 503 / 504 или timeout	Unknown, если контракт не даёт отдельной гарантии.
Документированный финальный отказ	FAILED только при доказанном окончательном отказе.

Непринятие второй попытки не разрешает неизвестность первой. Ошибка lookup не определяет результат платежа.

Закрывайте unknown по доказательству.

Подтверждение	Решение
Проверенные финальные успех / отказ	Применить финальный переход; credit gate сохраняется.
Нет строки / NOT_FOUND	Сохранить unknown, guard и владельца.
Финальный NOT_EXECUTED по PA-08	FAILED, если покрыты все попытки и запрещено будущее исполнение.

> **Ежедневная сверка: 02:00 UTC. Эскалация через 24 часа; инцидент через 7 дней. Возраст не создаёт финансовый результат.**

PA-08 требует окончательного доказательства неисполнения всех попыток и запрета будущего исполнения. Это дополнительная возможность провайдера. Отсутствие строки не доказывает отказ.

Новый ключ тоже требует проверки.

> **Те же пользователь + инструмент + сумма + валюта; платёж не разрешён → 409 с полным статусом и SHOW_EXISTING_PAYMENT.**

Сначала проверить принятый ключ. Guard нового ключа, создание и разрешение сериализованы. Через 6 минут и 24 часа unknown всё ещё блокирует. Шесть минут: пример возраста, не таймер.

Неотправленный CREATED может быть закрыт по WR-01. Отправленному или неоднозначному платежу нужно финальное подтверждение. Другие суммы/инструменты и новые намерения после успеха требуют отдельной политики.

У разбора несколько причин.

review_status объединяет отдельные причины разбора. Закрытие одной не должно удалять остальные.

Пришёл поздний проверенный успех	Ожидаемое поведение
Открыт только RESULT_OVERDUE	SUCCEEDED; закрыть просрочку; запланировать зачисление.
Просрочка + SECURITY_HOLD	SUCCEEDED; закрыть только просрочку; удержать зачисление.
Уже сохранён финальный отказ	Сохранить финал; открыть разбор противоречия.

Агрегат: NONE → OPEN → RESOLVED; новое основание может открыть его снова. У каждой причины есть владелец, следующее действие, доказательства и журнал.

Сохраняйте доверенные данные, включая отказы.

```
HMAC-SHA256(timestamp + "." + raw_body)
Delivery age: ±300s; body limit: 64 KiB
Business dedup: provider + event_id
```

Условие	Ответ / сохранение
Неверная подпись / неизвестный ключ	401; security metadata/хэш, без бизнес-изменений.
Подпись верна, timestamp устарел	401 после аутентифицированного карантина; доверенный lookup.
Подпись верна, нет event_id	400 после карантина; без применения; доверенный lookup.
Доверенное расхождение суммы/валюты	2xx после карантина; без применения; связанный разбор.
Валидное событие / сбой хранения	2xx после надёжного inbox / 503.

Проверять исходные байты, сравнивать подписи за постоянное время. Карантин зашифрован и отделён от inbox. Хэш тела дедуплицирует карантин, но не заменяет event_id.

Secret manager; ротация key ID с предыдущим ключом ≤24h. Компрометация: немедленный отзыв, восстановление через lookup. Inbox/карантин ≥30 дней и до закрытия связанных операций/разборов.

Зачисление тоже бывает неизвестным.

Успех без блокировки создаёт PENDING и outbox локально. Разрешение фиксирует DISPATCHING под общей блокировкой платежа. Атомарная дедупликация баланса обеспечивает одну проводку по payment_id, включая конкурентную доставку.

> **Ответ потерян, затем hold: OUTCOME_UNKNOWN остаётся неизвестным. Новые write остановлены; lookup чтением по payment_id продолжается.**

Подтверждённая проводка даёт POSTED даже при hold. NOT_FOUND не доказывает отсутствие зачисления: прежнее разрешение ещё может исполниться. REVIEW_REQUIRED требует блокировки и отсутствия возможного прежнего эффекта.

Без hold безопасно повторяется то же задание/payment_id. При hold запрещён даже идемпотентный write, способный создать проводку. Строгий отзыв после разрешения требует протокола баланса.

Сверяйте и продолжайте доставку.

Lookup выполняет Balance Recovery: +5s от неоднозначности, каждые 30s до 2m, затем каждые 5m. Проверяются ID проводки, получатель, сумма, валюта. Через 24h от успеха: Balance Operations; задания сохраняются.

Timeout write/lookup 10s; общие лимиты 10 вызовов/s и 10 одновременно. Backoff с jitter и circuit breaker; задания сохраняются. Целевой SLO: 99,9% за 60s, сигнал через 2m от успеха; не измерен.

Ровно одна проводка в итоге требует снятия hold, корректных данных, сохранённых заданий, обработки всех готовых задач и восстановления баланса/БД. Никогда не больше одной: safety. Знаменатель SLO и исключения: спецификация.

Чтение очереди ещё не разрешение.

Перед отправкой: блокировка строки платежа, перечитывание hold, commit DISPATCH_AUTHORIZED. Изменение hold использует ту же строку.

Порядок commit	Последствие
Hold раньше разрешения	Баланс не вызывается, даже если задание уже прочитано.
Разрешение раньше hold	Эта попытка может исполниться. Hold запрещает следующие.
Первый ответ потерян, hold нет	Повторить прежний payment_id; вернуть или создать одну проводку.
Ответ потерян, затем hold	OUTCOME_UNKNOWN; lookup продолжается, новые write запрещены.

> **Атомарность outbox локальная. БД баланса фиксирует проводку отдельно и обеспечивает её уникальность.**

Логическая граница начала: commit разрешения. Последующий hold не гарантирует отзыв вызова. Для строгого отзыва нужен протокол баланса, а не ещё одно локальное чтение.

Второй обработчик перечитывает.

Одновременные подтверждения	Требуемый результат
Lookup PENDING + webhook успеха	SUCCEEDED; старый pending не понижает финал.
Успех из обоих каналов	Один переход, одно логическое задание, одна проводка.
Финальные успех и отказ	Первый допустимый финал сохраняется; разбор конфликта.
Timeout + проверенный успех	Успех сохраняется при любом порядке commit.

Сохранить подтверждение. Проверить версию. Атомарно записать переход и outbox. При конфликте перечитать операцию и заново оценить то же подтверждение.

Локальная блокировка не отменяет начатый сетевой вызов. Пересечение вызовов также требует конкурентной идемпотентности провайдера.

Проверьте неудобные сценарии.

Сценарий	Критерий приёмки
AC-03 · Оба create дали timeout	Unknown сохраняется; третьего create нет.
AC-09 · Поздний успех при security hold	Успех сохранён; закрыта только просрочка; зачисление удержано.
AC-18 · Проводка есть, ответ потерян	Повтор возвращает ту же проводку.
AC-22 · Разные ключи одновременно	Одна операция; второй запрос получает 409.
AC-43 · Потерян ответ баланса + hold	Неизвестное зачисление, без нового write, lookup активен.

> **В спецификации 58 проверочных сценариев. AC ID связывают ожидаемые результаты с правилами.**

Это ожидаемые результаты, а не выполненные тесты реализации. Условия провайдера, конкурентность и SLO ещё нужно проверить на реальной системе.

Beyond the Scope

Процесс	Что не описано в этом кейсе
3DS; authorization / capture	Действия пользователя, авторизация и capture.
Refund; reversal; chargeback	Отдельные финансовые операции после успеха платежа.
Смена провайдера; части; FX	Дубликаты между провайдерами, остатки и конвертация.
Адаптеры отчётов провайдера	Форматы и доказательства окончательности.

Отдельно: исправление финалов, строгий отзыв в балансе и дубликаты между инструментами или после финального успеха.

Полная спецификация – см. GitHub.

GitHub Repository for the Inside the Spec

Yulia Melekhova on LinkedIn